Introduction to Machine Learning

Lab 1: Introduction

Classifying Iris Species with k-Nearest Neighbors

Student:	Your Name
Student ID:	Student ID
Instructor:	Zhang Li & Jing Ming
Institution:	Shandong Women's University
Date:	2026.09.23

Based on `notebooks/01-introduction-code.ipynb`. Replace all placeholder text before submission.

Contents

1 Objectives **2**

2 Background **2**

3 Dataset **2**

4 Methodology **3**

5 Experiments and Results **3**
5.1 Exercise 1 — Load the data 3
5.2 Exercise 2 — Explore the data 3
5.3 Exercise 3 — Train/test split 4
5.4 Exercise 4 — Build and train the classifier 4
5.5 Exercise 5 — Predict a new flower 4
5.6 Exercise 6 — Evaluate on the test set 4
5.7 Exercise 7 (bonus) — Does k matter? 4

6 Discussion **4**

7 Conclusion **5**

A Source Code **5**

1 Objectives

In this first lab you rebuild the book's very first machine learning application step by step. By the end you should be able to:

- Load a built-in scikit-learn dataset and inspect it.
- Split data into **training** and **test** sets.
- Build a **k-Nearest Neighbors** classifier.
- Use the `fit` → `predict` → `score` workflow to train, predict, and evaluate a model.

2 Background

In *supervised learning* a model learns from labelled examples $(\mathbf{x}_i, y_i)$. Because each iris belongs to one of three species, this is a **multiclass classification** problem (the possible answers are *classes*; the answer for one flower is its *label*).

To estimate how well a model **generalises** to unseen data, we hold out a **test set**: the model is trained only on the training set and evaluated on the test set. Accuracy summarises performance:

$$\text{accuracy} = \frac{\#\text{ correct predictions}}{\#\text{ samples}}.$$

k-Nearest Neighbors (k-NN). To classify a new point, k-NN looks at the k closest training points and takes a majority vote. With `n_neighbors=1` the prediction is simply the label of the single nearest training point. Every scikit-learn model follows the same pattern: `fit` (train), `predict` (classify new data), `score` (accuracy).

3 Dataset

We use the classic **Iris** dataset, bundled with scikit-learn as `load_iris()`. Each flower is described by four measurements, and the target is one of three species.

Table 1: Dataset summary (fixed facts, provided for reference).

Property	**Value**
Source	`sklearn.datasets.load_iris`
Task type	Multiclass classification (3 classes)
Number of samples	150
Number of features	4 (sepal/petal length & width, cm)
Classes	setosa, versicolor, virginica (50 each)
Split	`train_test_split(..., random_state=0)`
Training samples	112
Test samples	38

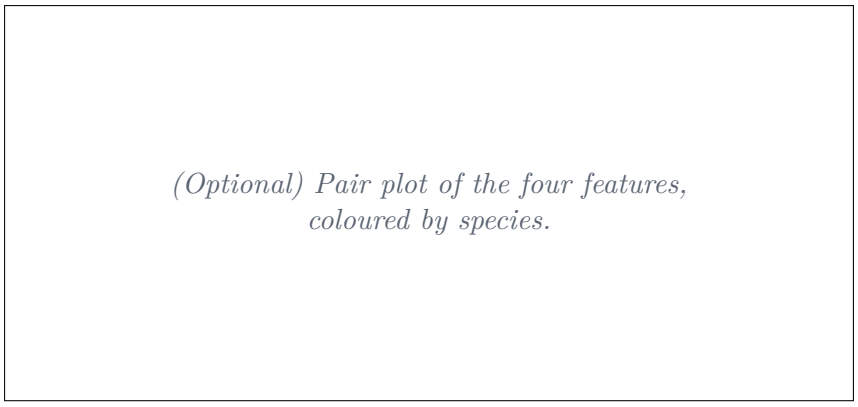


Figure 1: Scatter-matrix of the Iris features (optional "look at your data" step).

4 Methodology

The notebook is organised as seven short exercises built on this setup:

Listing 1: Imports (Setup).

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

1. **Load the data** into `iris` with `load_iris()`.
2. **Explore** the shape and the per-class counts.
3. **Split** into train/test with `random_state=0`.
4. **Build & train** `KNeighborsClassifier(n_neighbors=1)`.
5. **Predict** the species of a new flower `[5.0, 2.9, 1.0, 0.2]`.
6. **Evaluate** accuracy on the test set with `score`.
7. **(Bonus)** Vary k from 1 to 10 and compare test accuracy.

5 Experiments and Results

5.1 Exercise 1 — Load the data

Report what the dataset contains.

- Feature names: *TODO*
- Target names: *TODO* (expected setosa, versicolor, virginica)
- Data shape: *TODO* (expected $(150, 4)$)

5.2 Exercise 2 — Explore the data

- Samples, features: *TODO*
- Per-class counts: *TODO* (expected 50 / 50 / 50)

5.3 Exercise 3 — Train/test split

- `X_train` shape: *TODO* (expected 112 samples)
- `X_test` shape: *TODO* (expected 38 samples)

5.4 Exercise 4 — Build and train the classifier

State the model you trained and confirm it fitted successfully.

- Model: `KNeighborsClassifier(n_neighbors=1)`
- Fitted on: *TODO* training samples

5.5 Exercise 5 — Predict a new flower

For the new measurements `[5.0, 2.9, 1.0, 0.2]`:

- Predicted class index: *TODO*
- Predicted species name: *TODO* (expected *setosa*)

5.6 Exercise 6 — Evaluate on the test set

- Test accuracy: *TODO* (expected ≈ 0.97, must be > 0.90)

5.7 Exercise 7 (bonus) — Does k matter?

Record the test accuracy for each number of neighbours.

Table 2: Test accuracy as the number of neighbours k varies.

k	1	2	3	4	5	6	7	8	9/10
Test acc.	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*	*TODO*

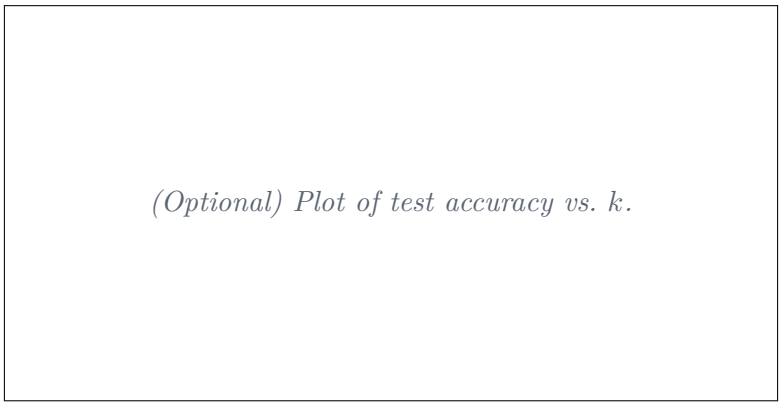


Figure 2: Test accuracy as a function of the number of neighbours (optional).

6 Discussion

Answer with evidence from your results above:

- **Generalisation.** Why do we evaluate on a separate test set instead of on the training data?

- **Prediction (Ex. 5).** Does the predicted species for the new flower seem reasonable given its small petals?
- **Choice of k (Ex. 7).** Which value(s) of k gave the best test accuracy? How does k relate to model complexity (small k vs. large k)?
- **Limitations.** What are the drawbacks of k-NN noted in the course (speed, many features, scaling)?

7 Conclusion

References

[1] A. C. Müller and S. Guido, *Introduction to Machine Learning with Python*. O'Reilly Media, 2016.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

A Source Code

Full code: `notebooks/01-introduction-code.ipynb`. Reference solution:

Listing 2: End-to-end reference solution.

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()                                          # Ex1
print(iris.data.shape, np.bincount(iris.target))            # Ex2

X_train, X_test, y_train, y_test = train_test_split(        # Ex3
    iris.data, iris.target, random_state=0)

knn = KNeighborsClassifier(n_neighbors=1)                   # Ex4
knn.fit(X_train, y_train)

X_new = np.array([[5.0, 2.9, 1.0, 0.2]])                    # Ex5
print(iris.target_names[knn.predict(X_new)][0])             # -> setosa

print(round(knn.score(X_test, y_test), 3))                  # Ex6
    (~0.974)

for k in range(1, 11):                                      # Ex7
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    print(k, round(m.score(X_test, y_test), 3))
```